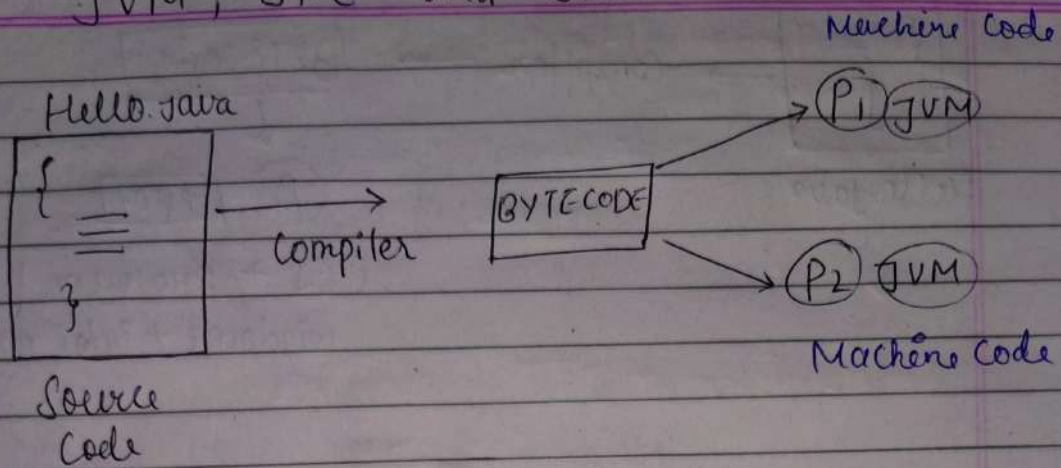


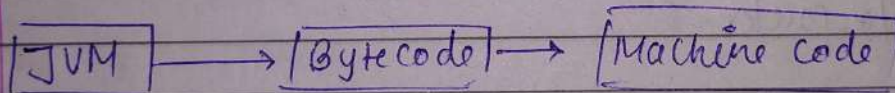
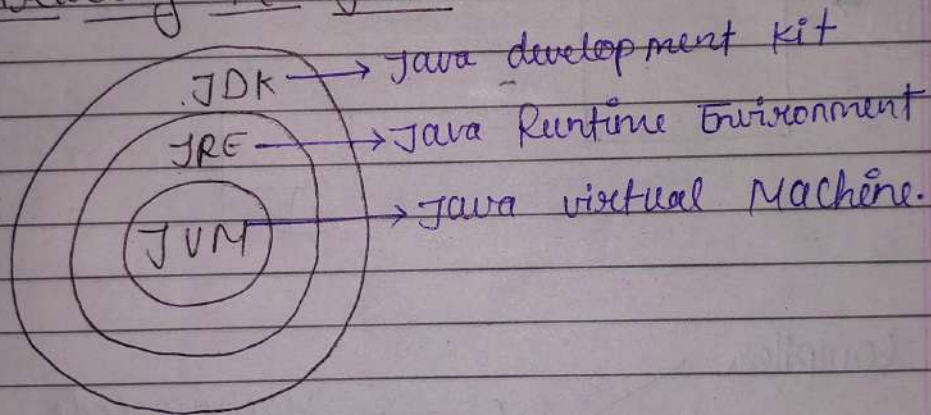
DAY-02.

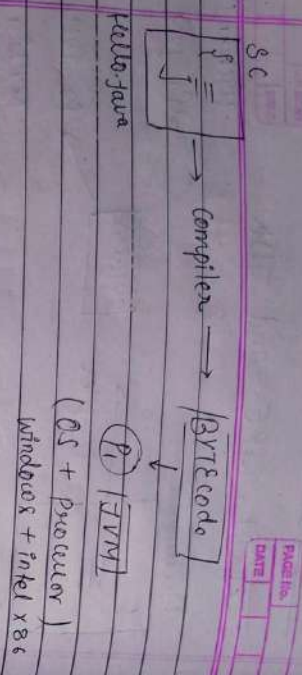
JVM, JRE and JDK

①

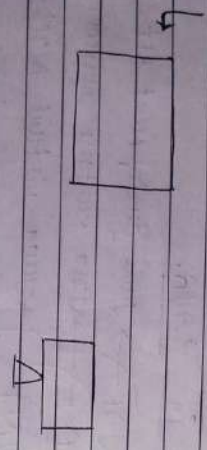


② Hierarchy in JAVA.

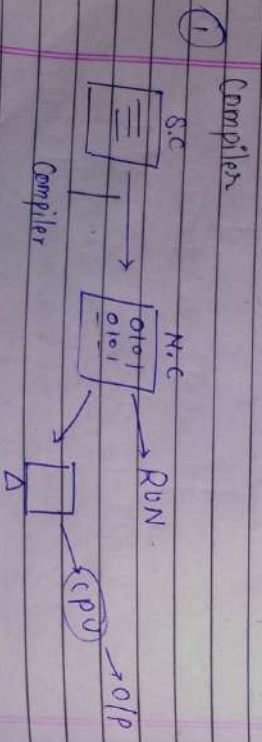




③ JVM

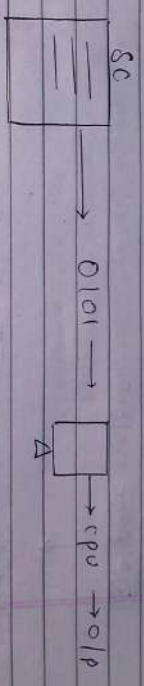


① Compiles
② Interpreter
S.C → Machine code



→ Yaha Compiler pura S.C file ko read kr
kai ek bath MC mai convert krn deta hai.
→ Convert code all at once

② Interpreter.



→ Convert code line by line

④ C/C++ → compiled language → M.C

python → interpreter → M.C

java → (compiled + interpreter) both

③ JVM

Bytecode → Interpreter → M.C
30's times

⑥ Endless problems

- ① Hardware slow
- ② RAM limited
- ③ Disc slow



Interpreter + JIT compiler

Just in time

Java → Compiler → Bytecode

JVM (Interpreter + JIT)

MC

⑦ JVM

Bytecode

↓ Frequent use of code

JIT

Interpreter

MC

↓ c/p

PAGE No.
DATE

JVM

- ① convert BYTECODE to MC
(Interpreter + JIT Compiler)

② Security (Sandbox Model)

Bytecode



JVM

"Bytecode Run inside JVM
where JVM provide secure
environment to it"

③ Garbage collection

② JRE

↳ JVM + Class
Library

- SC
- ① code (c/p)
 - ② File read
 - ③

PAGE No.
DATE

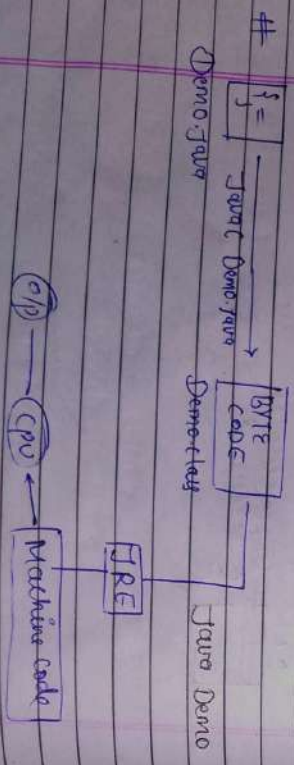
③ JDK (Java development kit).

JRE + Compiler,
Debugger,
JavaDoc

- ① JSE - Java standard edition
- ② JEE - Java Enterprise edition
- ③ JME - Java Micro edition

- ① JSE → Core Java
- ② JEE → Transactional, = (Banking etc)
- ③ JME - Lite weight edition.

How to run file of Java?



- ① JDK
- It is a software development kit used to build Java application.
- It contains the JRE and a set of development tools

Including (components)

- Compiler (Javac), debugger, and utility classes Javadoc and javadoc.
- Provide JRE, so it also takes care of running Java programs.

JDK is only for development (it is not needed for running Java programs).

→ It is platform dependent.

Working of JRE

- ① Source code (.java) - Developer write a Java program
- ② Compilation → The JDK's compiler (Javac) convert the code (.sc) into Bytecode (.class)
- ③ Execution - The JVM executes the bytecode, translating it into native instruction.

② JRE (Java Runtime Environment)

- It provides an environment to run the Java programs but does not include development tools.
- It is intended for end-users who only need to execute application.

Containing:

- ① JVM and standard libraries
- ② provide all runtime requirement for Java applications
- ③ Does not support compilation or debugging.

Note

JRE is only for running application, not for developing
- platform-dependent).

Working:

- ① Class Loading - Load compiled (.class) files into memory.

- ② Bytecode verification: Ensure security & validity of bytecode.

- ③ Execution - use JVM (Interpreter + JIT compiler) to execute instruction and make system calls.

③ JVM

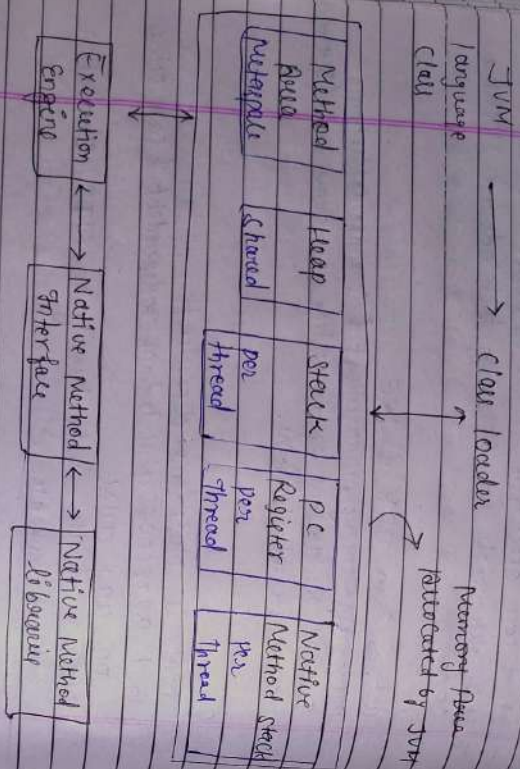
- It is core execution engine of Java.
- It is responsible for converting bytecode into machine code.

- part of both JDK & JRE
- perform memory management & garbage collection.
- provide portability by executing the same code (bytecode) on different platform.

Note

- ① JVM is platform dependent
- ② But Bytecode is platform independent & can run on any JVM.
- ③ Modern JVMs rely heavily on (JIT) compilation for performance.

Working of JVM



- ① Loading - Class loader loads the byte code into memory.
- ② Linking - perform verification, preparation & resolution.
- ③ Initialization - Execute class constructors & static initialization.
- ④ Execution - Interpreter or compiler byte code into native code.

(1) M1

Here JVM stores the class data.

- ① Method code that you run after compilation.
- ② Variables used to pass data to other M1.
- ③ Static data.

- It shared among all threads.

- ② Heap (Shared)
 - ① Has objects are stored.
 - ② When you use new keyword - this create an object and it store at heap.

③ Stack

① Every thread have separate stack.

- ② It store
 - ① Local variable
 - ② Function call.

- Whenever function call occurs - It create entry in stack.
- function finished - entry delete from stack.

④ PC Register

It tells the current location of program (main abhi kounsa program ka instruction run ho raha hai).

PAGE No.
 DATE

⑤ Native Method stack (per thread).

Whenever Java call any code of C/C++, then its data (C/C++) store at here.

⑥ Execution engine

- This part actually run the code

- Byte code — Machine code.

It have

① JIT

② Interpreter.

⑦ Native Method interface (JNI)

Java

It allow¹ to use C/C++ libraries.

⑧ Native Method libraries

→ These are actual external libraries (C/C++/etc)